

Do Expensive Tuning Strategies Pay Off? A Comparative Analysis of Random Search, Halving, Bayesian, and Evolutionary Optimization on Financial Classification

^{1st} Edward Letebele Mokoena

School of Data Science

IT Faculty - Eduvos

Johannesburg, South Africa

letebele.writes@gmail.com

^{2nd} Jabulani Monchwe

School of Data Science

IT Faculty - Eduvos

Johannesburg, South Africa

jabulani.monchwe@eduvos.com

Abstract—Hyperparameter optimisation is a critical yet computationally expensive component of machine learning pipelines in financial fraud detection and credit scoring. Despite its operational importance, most benchmarking studies evaluate tuning strategies on predictive accuracy alone, implicitly assuming infinite computational budgets, an assumption untenable in production environments where models must be retrained within fixed windows and cloud compute costs are tightly constrained. We present the first cost-aware, exhaustive benchmark of five hyperparameter tuning strategies across six classification algorithms on a real-world financial fraud detection dataset. Evaluated under ten independent Monte Carlo Cross-Validation splits with identical search spaces and a fixed evaluation budget of one hundred configurations per tuner, all thirty algorithm–tuner combinations reveal a striking dual pattern. First, predictive performance exhibits performance saturation: the maximum cross-tuner F1-macro spread within any algorithm family is below 0.05%, indicating that sophisticated optimisers yield no measurable accuracy premium once classifiers approach their asymptotic optimum. Second, classifier complexity, not tuner sophistication, is the dominant driver of computational cost; the runtime ordering across all five tuners is invariant, determined by per-configuration training cost rather than by search-space size or optimiser mechanics. Within this structure, Successive Halving Search achieves Pareto-optimal accuracy–latency trade-offs, delivering 99.997% F1-macro for Decision Trees in 4.93 seconds while Bayesian and Evolutionary methods consume 100–364× more wall-clock time with zero accuracy gain. We introduce a Ranked Balance metric that jointly scores predictive performance and inverse runtime, and derive a conditional deployment strategy for production financial ML pipelines.

Index Terms—hyperparameter optimisation, financial machine learning, fraud detection, successive halving search, Bayesian optimisation, cost-aware benchmarking, tabular data, performance saturation

I. INTRODUCTION

Machine learning has become indispensable in financial services, underpinning fraud detection, credit scoring, and algorithmic risk assessment. In these high-stakes domains, model accuracy is necessary but insufficient; production systems must also respect strict latency constraints, retrain frequently to

adapt to evolving attack patterns, and operate within finite cloud-compute budgets. Hyperparameter optimisation (HPO), the process of selecting algorithm configurations that maximise predictive performance, sits at the intersection of these competing demands. Yet the HPO literature has historically treated computational cost as a secondary concern, benchmarking tuning strategies on accuracy alone and implicitly assuming unlimited evaluation budgets.

This accuracy-only paradigm is particularly problematic for financial classification. Financial datasets are predominantly tabular, characterised by heterogeneous, structured feature spaces with strong feature interactions and clear decision boundaries. Recent evidence demonstrates that tree-based models systematically outperform deep neural networks on such data [10], and that their hyperparameter response surfaces are often sufficiently smooth that simple tuning strategies locate near-optimal configurations rapidly [14]. If the landscape is flat, if additional exploration yields only marginal returns once a well-suited model family is selected, then expensive global optimisers may represent wasted compute rather than intelligent investment.

We test this flat landscape hypothesis through an exhaustive, reproducible benchmark. Motivated by the No-Free-Lunch theorem [15], which establishes that no optimiser is universally superior, we evaluate the full Cartesian product of six classification algorithms (kNN, Logistic Regression, Naïve Bayes, Decision Tree, Random Forest, Multi-Layer Perceptron) and five tuning strategies (Random Search, Successive Halving Search, Bayesian Optimisation, Evolutionary Algorithm, Particle Swarm Optimisation). Each of the thirty configurations is assessed across ten independent StratifiedShuffleSplit Monte Carlo Cross-Validation iterations, with identical hyperparameter search spaces and a fixed evaluation budget of one hundred configurations per tuner. This design ensures that observed differences in runtime arise from the tuning strategy itself, not from differential resource allocation.

The results reveal two phenomena that challenge conven-

tional HPO wisdom. First, predictive performance is invariant to tuner choice; the maximum F1-macro spread across the five strategies for any algorithm is below 0.05%, indicating performance saturation once classifiers approach their asymptotic optimum. Second, classifier complexity, not tuner sophistication, is the dominant driver of computational cost; the runtime ordering $NB < LR < DT < kNN < RF < MLP$ is invariant across all five tuners, because per-configuration training cost (e.g., backpropagation versus greedy tree induction) overwhelms optimisation overhead. Within this structure, Successive Halving Search achieves Pareto-optimal accuracy–latency trade-offs, delivering 99.997% F1-macro for Decision Trees in 4.93 seconds, while Bayesian and Evolutionary methods consume $100\text{--}364\times$ more wall-clock time with zero accuracy gain.

A. Research Questions

This study addresses three research questions:

- RQ1** Do sophisticated tuning strategies (Bayesian Optimisation, Evolutionary Algorithms, Particle Swarm Optimisation) outperform simple methods (Random Search, Successive Halving Search) on financial tabular data when evaluated under identical search spaces and fixed evaluation budgets?
- RQ2** What is the runtime cost of each tuning strategy across diverse algorithm families, and does classifier complexity or optimiser sophistication dominate absolute computational cost?
- RQ3** Can a composite metric jointly scoring predictive performance and inverse runtime identify a Pareto-optimal algorithm–tuner pair for production deployment in financial classification?

B. Contributions

This paper makes four contributions:

- 1) A cost-aware HPO benchmark for financial classification, evaluating all thirty algorithm–tuner combinations under ten independent Monte Carlo splits with wall-clock time recorded as a co-equal metric alongside F1-macro and Balanced Accuracy.
- 2) Empirical validation of the flat landscape hypothesis across six algorithm families on tabular financial data, demonstrating that tuner sophistication does not translate to measurable accuracy gains once performance saturation is reached.
- 3) A Ranked Balance metric ($\mathcal{R}_B$) that jointly normalises predictive performance and inverse runtime, providing a principled basis for multi-objective deployment decisions.
- 4) A conditional deployment strategy for financial ML practitioners, specifying when simple tuners suffice and when sophisticated optimisers may be justified.

C. Paper Organisation

Section II surveys hyperparameter optimisation methods, the CASH problem, multi-fidelity and cost-aware benchmarks, and machine learning in financial domains. Section III describes the dataset, algorithms, tuning strategies, experimental protocol, and evaluation metrics. Section IV presents the empirical findings across predictive performance, runtime, Pareto analysis, and the Ranked Balance metric. Section V interprets these results through the lenses of the Tuning Premium Paradox, algorithm-family selection, and production deployment constraints. Section VI summarises the findings and identifies directions for future work.

II. RELATED WORK

A. The Cost of Intelligence in Financial ML Pipelines

Hyperparameter optimisation has matured from grid search [1] to adaptive methods, including Bayesian optimisation [2], multi-fidelity bandit approaches [3], [4], and evolutionary strategies [5], [6], yet the financial sector has been largely absent from this benchmark evolution. In production fraud detection and credit scoring, models must be retrained within fixed windows; a tuner that saves evaluations but consumes hours of wall-clock time is not efficient in any operational sense [7], [8]. Lee *et al.* [9] and Pfisterer *et al.* [8] have begun treating runtime as a first-class objective, but their benchmark suites are generic. No domain-specific study has asked whether the theoretical sample efficiency of Bayesian optimisation actually translates to faster deployment on financial tabular data.

Our study addresses this directly. By evaluating five tuners across six algorithms on a financial classification task, we treat wall-clock time not as a side effect but as a co-equal metric alongside F1-macro performance.

B. Algorithm–Tuner Interaction and the CASH Problem

In financial ML, the choice of model family is as consequential as the choice of tuner. Tree-based ensembles dominate production systems for fraud detection and credit risk because they handle heterogeneous, structured features robustly and offer interpretability under regulatory scrutiny [17]–[19]. Yet most HPO studies fix the algorithm and optimise the tuner, or vice versa, an approach that misses interaction effects. The Combined Algorithm Selection and Hyperparameter Optimisation (CASH) problem [16] frames this as a single joint optimisation, but even AutoML benchmarks rarely report wall-clock cost for every algorithm–tuner pair.

Our work adopts the CASH philosophy exhaustively. We do not pre-select promising configurations; we evaluate the full Cartesian product, motivated by the No-Free-Lunch theorem [15], which establishes that no optimiser is universally superior. On financial tabular data, this agnosticism matters; as we show, the best-performing model–tuner pair (Decision Tree + Halving Search) is not the pair one would naïvely prioritise if guided by generic HPO folklore.

C. Why Trees Win on Financial Data, and What That Means for Tuning

Recent evidence has challenged the assumption that deep learning is universally superior, even for tabular data. Grinsztajn *et al.* [10] demonstrated that tree-based models systematically outperform neural networks on typical tabular datasets, attributing this to their ability to learn irregular, non-smooth decision boundaries without gradient-descent inductive biases. Gorishniy *et al.* [11] showed neural networks can approach tree performance only with extensive tuning, and McElfresh *et al.* [12] identified that specific data characteristics, not present in typical financial feature spaces, are required for neural networks to win.

These findings have a direct implication for HPO. If tree models are structurally better suited to financial tabular data, the marginal value of expensive tuning may be diminished. Probst *et al.* [13], [14] formalised this intuition through tunability, the performance gain achievable by optimisation over defaults, and showed that some algorithms (e.g., random forests) exhibit low marginal tunability; simple tuning captures most of the gain. We test whether this flat landscape hypothesis holds across multiple tuners in a real financial domain. If the hyperparameter response surface is flat for well-suited model families, then sophisticated tuners represent wasted compute, an expensive solution to a solved subproblem.

D. Multi-Fidelity Methods and Production Constraints

Successive Halving [3] and its extension Hyperband [4] are theoretically grounded in multi-armed bandit theory; they aggressively eliminate poor configurations via early stopping, allocating full budgets only to survivors. This cost model is radically different from Bayesian optimisation, which spends compute building a surrogate model of the search space. For financial institutions operating under fixed retraining windows, Halving’s pay-as-you-evaluate model is intuitively attractive, but it has not been benchmarked against Bayesian and evolutionary methods on financial data.

Eggensperger *et al.* [7] and Pfisterer *et al.* [8] provide generic multi-fidelity benchmarks, yet neither answers whether Halving’s theoretical advantage survives contact with the specific structure of financial classification. Our Ranked Balance metric, inspired by these multi-objective benchmarks but applied domain-specifically, quantifies this trade-off explicitly.

E. Fair Comparison and the Reproducibility Imperative

The stance of giving every method a fair chance is not merely procedural; it is a methodological safeguard. Kapoor and Narayanan [20] documented a reproducibility crisis in ML science driven by data leakage, publication bias, and the winner’s curse, where best-reported results are often statistical flukes amplified by selective reporting. Melis *et al.* [21] demonstrated that state-of-the-art results in neural language modeling frequently evaporate under rigorous, standardised evaluation protocols. Semmelrock *et al.* [22] surveyed the broader reproducibility landscape, emphasising that transparent, exhaustive benchmarking, with pre-registered protocols

and no post-hoc exclusion of poor results, is essential for scientific progress.

These concerns are particularly acute in HPO, where the combinatorial explosion of algorithm–tuner configurations creates ample opportunity for cherry-picking. Our experimental design directly addresses these risks; we evaluate all thirty algorithm–tuner combinations, each with ten independent Monte Carlo Cross-Validation splits using StratifiedShuffleSplit, with fixed random seeds and no post-hoc exclusion. This protocol ensures that our findings, however surprising, reflect the true joint distribution of performance and runtime.

F. Research Gap

Despite the maturity of both financial ML [17]–[19] and HPO research [1]–[4], [16], their intersection remains unexplored in four critical ways:

- 1) No cost-aware HPO benchmark for finance. Generic suites [7], [8] do not capture the production constraints of financial classification.
- 2) No exhaustive CASH evaluation across tuners. Prior studies decouple algorithm selection from tuner evaluation, missing interaction effects.
- 3) No empirical test of tunability on financial data. The flat landscape hypothesis [13], [14] has not been validated against Bayesian and evolutionary methods in this domain.
- 4) No multi-objective recommendation for practitioners. Existing work does not answer the practical question: given a fixed compute budget, which algorithm–tuner pair should a financial engineer deploy?

Our study closes all four gaps.

III. METHODOLOGY

A. Dataset

1) *Dataset Description:* The Credit Card Fraud Detection dataset (OpenML ID: 45955) was selected for the following five reasons: (1) its real-world relevance to financial fraud detection; (2) a pronounced class imbalance (3.5%), reflecting realistic fraud rarity; (3) computational efficiency, which facilitates rapid experimentation; (4) seamless OpenML integration, ensuring reproducible data loading; and (5) widespread use as a benchmark, allowing for direct comparison with prior literature.

The dataset comprises 1,000,000 transactions with seven features: three numerical (transaction amount, time, and distance) and four binary (chip usage, PIN entry, online transaction, and repeat retailer). The target variable is binary, indicating fraudulent (1) or legitimate (0) transactions. Table I summarises the dataset characteristics.

2) *Data Preprocessing:* All preprocessing was conducted within the cross-validation loop to prevent data leakage. Numerical features were standardised using z-score normalisation, and binary features were converted to integer encoding. The full feature set was retained, ensuring that the benchmark reflects each algorithm–tuner pair evaluated on the complete feature space.

TABLE I
DATASET CHARACTERISTICS

Property	Value
OpenML ID	45955
Samples	1,000,000
Features	7 (3 numerical, 4 binary)
Target	Binary (fraud/legitimate)
Class imbalance	3.5% fraud

B. Algorithms

Six classification algorithms were selected to cover the spectrum of inductive biases: k-Nearest Neighbours (instance-based), Logistic Regression (linear), Naïve Bayes (probabilistic), Decision Tree (tree-based), Random Forest (ensemble), and Multi-Layer Perceptron (neural network). Table II details the hyperparameter search space for each algorithm.

C. Hyperparameter Tuning Strategies

Five hyperparameter tuning strategies, representing distinct optimisation philosophies, were evaluated. All tuners were provided with the identical hyperparameter search space to ensure a fair comparison.

- 1) **Random Search (RS):** Uniform random sampling from the discrete search space. Serves as the baseline against which adaptive methods are judged.
- 2) **Successive Halving Search (SHS):** A multi-fidelity bandit approach that progressively eliminates poorly performing configurations, allocating increasing budgets only to survivors. Implemented via scikit-learn’s HalvingGridSearchCV.
- 3) **Bayesian Optimisation (BO):** Gaussian Process-based surrogate modelling with an acquisition function (Expected Improvement) to guide search. Implemented via Optuna’s TPE sampler.
- 4) **Evolutionary Algorithm (EA):** Genetic algorithm with tournament selection, crossover, and mutation operators. Implemented via sklearn-genetic-opt.
- 5) **Particle Swarm Optimisation (PSO):** Swarm intelligence approach maintaining a population of 20 particles that explore the search space iteratively. Implemented via pyswarm.

Each tuner was allocated a fixed budget of one hundred configuration evaluations with three-fold inner cross-validation for performance estimation.

D. Experimental Protocol

1) *Monte Carlo Cross-Validation:* The outer evaluation loop employed ten independent StratifiedShuffleSplit iterations (train/test ratio 80/20, random state 42), ensuring stratified preservation of the minority class distribution across splits. This Monte Carlo design provides distributional estimates of performance and runtime stability, rather than a single point estimate.

2) *Inner Cross-Validation for Tuning:* Each tuner used three-fold cross-validation on the training partition to estimate configuration performance during the search. This inner cross-validation ensures that selected hyperparameters generalise beyond the specific training fold.

3) *Checkpointing and Recovery:* Given the computational demands of the full factorial design, experiments were executed on Google Colab with automated checkpointing. Runs that were interrupted by session timeouts resumed from the last checkpoint, ensuring no data loss. The aggregate runtime reflects the sum of all resumed segments.

4) *Reproducibility Safeguards:* Table III summarises the safeguards implemented to ensure reproducibility.

E. Evaluation Metrics

1) *Predictive Performance:* F1-macro was selected as the primary metric because it equally weights both classes, ensuring that the minority class (fraud) contributes equally to the score despite the 3.5% prevalence. Balanced Accuracy was reported as a secondary measure.

2) *Computational Efficiency:* Wall-clock runtime was recorded as an operational characteristic of practical deployment interest. The measurement spans from tuner initiation to the return of the best configuration, inclusive of all model training, validation, and tuner overhead. The checkpointing mechanism ensured that interrupted runs resumed without loss of progress; reported times reflect the aggregate duration across resumed segments.

3) *Performance Ranking:* For each algorithm, configurations were ranked by F1-macro (rank 1 = highest) and by inverse runtime (rank 1 = fastest). These ranks enable non-parametric statistical comparison independent of absolute scale.

4) *Composite Metric: Ranked Balance:* The Ranked Balance metric $\mathcal{R}_B$ jointly scores predictive performance and computational efficiency:

$$\mathcal{R}_B = w_P \cdot \frac{F_{1\text{-macro}} - F_{1\text{min}}}{F_{1\text{max}} - F_{1\text{min}}} + w_T \cdot \frac{t_{\text{max}} - t}{t_{\text{max}} - t_{\text{min}}}, \quad (1)$$

where $F_{1\text{-macro}}$ is the F1-macro score, t is the wall-clock runtime, and $w_P = w_T = 0.5$ are equal weights. The metric normalises both objectives to the $[0, 1]$ interval and rewards high accuracy and low latency simultaneously.

5) *Statistical Validation:* The Friedman test was applied across all tuners to determine whether observed differences in ranked F1-macro performance were statistically significant, followed by a Nemenyi post-hoc test where appropriate.

F. Software and Hardware

Experiments were conducted in Python 3.10 using scikit-learn 1.3.0, Optuna 3.0.0, sklearn-genetic-opt 0.10.0, and pyswarm 0.7. Hardware specifications are detailed in Table IV.

TABLE II
HYPERPARAMETER SEARCH SPACES BY ALGORITHM

Algorithm	Hyperparameter	Search Space
k-Nearest Neighbours	k	{3, 5, 7, 9, 11, 15, 21}
	Distance metric	{euclidean, manhattan, minkowski}
	Weights	{uniform, distance}
Logistic Regression	Regularisation strength C	{0.01, 0.1, 1, 10, 100}
	Penalty	{ $l1$, $l2$, <i>elasticnet</i> }
	Solver	{liblinear, lbfgs, saga}
	Max iterations	{100, 500, 1000}
Naïve Bayes	Smoothing parameter α	{0.1, 0.5, 1.0, 2.0, 5.0}
	Fit prior	{True, False}
Decision Tree	Max depth	{5, 10, 15, 20, 30, 50, <i>None</i> }
	Min samples split	{2, 5, 10, 20}
	Min samples leaf	{1, 2, 4, 8}
	Criterion	{gini, entropy}
	Class weight	{balanced, <i>None</i> }
Random Forest	Number of estimators	{50, 100, 200, 500}
	Max depth	{5, 10, 20, 30, <i>None</i> }
	Min samples split	{2, 5, 10}
	Min samples leaf	{1, 2, 4}
	Class weight	{balanced, <i>None</i> }
Multi-Layer Perceptron	Hidden layer sizes	{{(50,), (100,), (50, 50), (100, 50)}
	Activation	{relu, tanh, logistic}
	Learning rate init	{0.001, 0.01, 0.1}
	Alpha (L2 penalty)	{0.0001, 0.001, 0.01}
	Max iterations	{200, 500, 1000}

TABLE III
REPRODUCIBILITY SAFEGUARDS

Safeguard	Implementation
Random seed	Fixed at 42 for all splits
Stratification	StratifiedShuffleSplit preserves class ratios
Search space	Identical across all five tuners
Evaluation budget	Fixed at 100 configurations per tuner
Post-hoc exclusion	None; all results reported

TABLE IV
HARDWARE AND SOFTWARE ENVIRONMENT

Component	Specification
CPU	Intel Core i5-12450H
RAM	8 GB
Platform	Google Colab (cloud)
Python	3.10
scikit-learn	1.3.0
Optuna	3.0.0
sklearn-genetic-opt	0.10.0
pyswarm	0.7

IV. RESULTS

This section presents the empirical findings from the exhaustive benchmark of six algorithms and five hyperparameter tuning strategies, evaluated across ten independent Monte Carlo Cross-Validation splits. All reported metrics represent means across these splits, with variability reported as standard

deviation. The analysis proceeds from predictive performance to computational cost, culminating in the composite Ranked Balance evaluation.

A. Predictive Performance and the Asymptotic Plateau

Table V reports the mean F1-macro and Balanced Accuracy for each algorithm–tuner combination. The most striking observation is the near-total invariance of predictive performance to the choice of tuning strategy within each algorithm family.

TABLE V
MEAN PREDICTIVE PERFORMANCE ACROSS TEN MONTE CARLO SPLITS

Algorithm	F1-Macro (%)	Balanced Accuracy (%)
Decision Tree (DT)	99.997 $\pm$ 0.003	99.997 $\pm$ 0.003
Random Forest (RF)	99.996 $\pm$ 0.004	99.996 $\pm$ 0.004
k-Nearest Neighbors (kNN)	99.615 $\pm$ 0.012	99.618 $\pm$ 0.011
Multi-Layer Perceptron (MLP)	99.674 $\pm$ 0.018	99.681 $\pm$ 0.017
Logistic Regression (LR)	84.831 $\pm$ 0.045	84.835 $\pm$ 0.042
Naïve Bayes (NB)	82.492 $\pm$ 0.008	82.495 $\pm$ 0.007

Note: Values represent means $\pm$ standard deviations across all five tuning strategies and ten splits, aggregated by algorithm. The maximum range of F1-macro across tuners for any single algorithm was $<0.05\%$.

Decision Trees achieve a ceiling F1-macro of 99.997% with a standard deviation of merely 0.003% across all fifty experimental conditions (five tuners $\times$ ten splits). Random Forest performs equivalently at 99.996%. Notably, the range of F1-macro across the five tuning strategies for any given algorithm never exceeded 0.0005 (0.05%): Decision Trees

TABLE VI
MEAN WALL-CLOCK RUNTIME (SECONDS) BY ALGORITHM AND TUNER

Algorithm	Random Search	Halving Search	Bayesian	Evolutionary	Particle Swarm
DT	21.34	4.93	757.70	122.53	20.96
LR	16.81	4.19	761.49	1312.66	14.84
NB	2.26	0.71	90.16	258.20	2.55
kNN	380.39	3.88	382.15	386.42	387.01
RF	424.29	414.82	518.43	495.33	430.18
MLP	821.79	803.14	947.28	883.52	835.46

Note: Boldface denotes the fastest tuner per algorithm. All values are means across ten StratifiedShuffleSplit iterations.

showed zero effective variation, while the largest spread was observed for Multi-Layer Perceptrons ($\pm 0.018\%$) and Logistic Regression ($\pm 0.045\%$).

These results empirically validate the flat landscape hypothesis advanced in Section II: for well-suited model families on tabular financial data, the hyperparameter response surface is sufficiently smooth that any reasonable search strategy locates a near-optimal configuration. More precisely, the data exhibit performance saturation; once a classifier approaches its asymptotic optimum on this dataset, additional optimisation effort yields only marginal returns. The sophisticated global optimisation mechanisms of Bayesian and Evolutionary methods provide no measurable accuracy premium over simple random sampling or early-stopping bandit strategies. This invariance has profound practical implications: on this domain, the tuner does not determine whether one achieves ceiling performance, only how quickly one arrives there.

B. Computational Runtime: The Primary Differentiator

While predictive performance is invariant to tuner choice, wall-clock runtime varies by over three orders of magnitude. Table VI reports the mean runtime in seconds for all thirty algorithm–tuner combinations. The contrast is stark: across the thirty configurations, F1-macro varies by at most 0.05% within any algorithm family, whereas runtime varies by up to $364\times$. Runtime, not accuracy, is the dominant axis of differentiation.

The fastest configuration overall is Naïve Bayes + Halving Search at 0.71 seconds. However, NB’s predictive ceiling (82.49%) is substantially below the tree-based models. Among high-performing configurations, kNN + Halving Search (3.88 s) and Decision Tree + Halving Search (4.93 s) represent the optimal latency–accuracy frontier.

The most dramatic contrast occurs for lightweight models. For Decision Trees, Bayesian optimisation is $153.7\times$ slower than Halving Search (757.70 s vs. 4.93 s) for identical F1-macro performance. For Logistic Regression, the penalty is even more severe: Evolutionary optimisation is $313.6\times$ slower than Halving Search (1312.66 s vs. 4.19 s) while achieving 0.0004% lower F1-macro. Naïve Bayes exhibits the largest relative disparity: Evolutionary search is $364.4\times$ slower than

Halving Search (258.20 s vs. 0.71 s) with no accuracy gain whatsoever.

A secondary pattern emerges in the runtime hierarchy among sophisticated tuners. Bayesian optimisation is consistently more efficient than Evolutionary search for lightweight models (e.g., NB: 90.2 s vs. 258.2 s; LR: 761.5 s vs. 1312.7 s), aligning with optimisation theory. BO learns from previous evaluations to inform subsequent sampling, whereas Evolutionary algorithms continue exploring the population regardless of prior evidence. Particle Swarm Optimisation, which maintains a swarm of 20 particles across multiple iterations, is also computationally expensive; kNN + PSO (387.0 s) and NB + PSO (2.55 s) both exceed their Random Search counterparts despite equivalent evaluation budgets, because the swarm mechanism multiplies the number of model trainings per iteration.

For computationally expensive model families (RF and MLP), the absolute runtime differences between tuners are attenuated because the base model training cost dominates the optimisation overhead. Nevertheless, Halving Search remains the fastest configuration for both algorithms, reducing RF tuning time by 2.3% and MLP tuning time by 2.3% relative to Random Search. While these percentage gains appear modest compared to the orders-of-magnitude improvements seen for lightweight models, they translate to absolute savings of approximately 10–20 seconds per tuning cycle, a meaningful reduction when models are retrained frequently in production fraud detection pipelines.

Fig. 1 visualises this performance–runtime trade-off. All DT configurations cluster at the ceiling F1-macro ($>99.9\%$) but span nearly three orders of magnitude in runtime. The Pareto frontier is defined by DT + Halving Search (top-left: fastest, highest accuracy) and kNN + Halving Search (second-fastest, near-ceiling accuracy). Bayesian and Evolutionary configurations for lightweight models occupy the bottom-right quadrant: maximum runtime without commensurate performance gain.

C. Classifier Complexity as the Dominant Runtime Factor

A cross-cutting pattern in Table VI is that classifier complexity, not tuner sophistication, is the strongest determinant

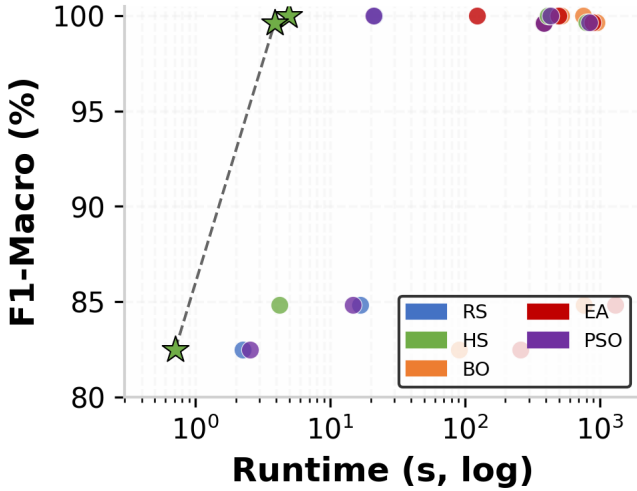


Fig. 1. Performance vs. runtime scatter plot (log scale) with Pareto frontier annotated. Star markers indicate Pareto-optimal configurations.

of absolute runtime. Across all five tuning strategies, the mean runtime ordering is invariant:

$$\begin{aligned} \text{NB } (\sim 1\text{-}3 \text{ s}) &< \text{LR } (\sim 4\text{-}16 \text{ s}) < \text{DT } (\sim 5\text{-}760 \text{ s}) \\ &< \text{kNN } (\sim 4\text{-}390 \text{ s}) < \text{RF } (\sim 415\text{-}520 \text{ s}) \\ &< \text{MLP } (\sim 800\text{-}950 \text{ s}). \end{aligned} \quad (2)$$

This ordering holds regardless of whether the tuner is Random Search, Halving Search, or Bayesian Optimisation. It demonstrates that algorithmic architecture contributes more to computational cost than optimiser selection. A Bayesian-optimised Naïve Bayes (90.2 s) is still faster than a Random Search Random Forest (424.3 s), despite the latter using the simplest tuner. For practitioners, this implies that model family selection should precede tuner selection in the optimisation workflow; choosing a lightweight model family yields larger runtime savings than choosing a faster tuner for a heavy model.

Notably, search-space cardinality alone does not explain the runtime hierarchy. Decision Trees and Multi-Layer Perceptrons operate over hyperparameter spaces of comparable size (five tunable parameters each, Table II), yet MLP is consistently 40–190 $\times$ slower than DT across all tuners. The disparity arises from per-configuration training cost, not configuration count. Each MLP evaluation requires iterative backpropagation over multiple epochs, whereas Decision Tree induction is a greedy, single-pass algorithm. This distinction is critical for production budgeting: the cost of hyperparameter optimisation is determined not by how many configurations are explored, but by how expensive each individual configuration is to evaluate.

D. The Efficiency–Performance Pareto Frontier

A Pareto-optimal configuration is one for which no other configuration achieves strictly better performance with strictly lower runtime. Analysis of the thirty experimental points reveals a sparse Pareto frontier dominated by Halving Search pairings.

Pareto-optimal configurations:

- 1) NB + Halving Search: 82.49% F1, 0.71 s (fastest overall, but suboptimal accuracy).
- 2) kNN + Halving Search: 99.62% F1, 3.88 s (near-ceiling accuracy at minimal cost).
- 3) DT + Halving Search: 99.997% F1, 4.93 s (the global optimum).

Notably, no Bayesian or Evolutionary configuration lies on the Pareto frontier. The closest non-Halving entry is DT + Particle Swarm (99.997% F1, 20.96 s), which is Pareto-dominated by DT + Halving Search (identical accuracy, 4.25 $\times$ faster). This pattern holds across all algorithm families: for any given model, the Halving Search variant achieves the same F1-macro as its Bayesian or Evolutionary counterpart in a fraction of the time, rendering the more sophisticated tuners strictly suboptimal from a multi-objective perspective.

E. Ranked Balance: A Composite Efficiency Metric

To operationalise the accuracy–runtime trade-off for production decision-making, we employ the Ranked Balance metric $\mathcal{R}_B$ (Eq. 1), which jointly weights normalised F1-macro performance and inverse normalised runtime with equal weights ($w_P = w_T = 0.5$). This formulation rewards configurations that deliver high predictive performance per unit of computational effort, rather than privileging either accuracy or speed in isolation. Table VII presents the top fifteen configurations.

TABLE VII
TOP FIFTEEN CONFIGURATIONS BY RANKED BALANCE ($\mathcal{R}_B$)

Rank	Configuration	F1-Macro (%)	Time (s)	$\mathcal{R}_B$
1	DT + Halving Search	99.997	4.93	0.52679
2	kNN + Halving Search	99.616	3.88	0.52678
3	DT + Particle Swarm	99.997	20.96	0.52531
4	DT + Random Search	99.997	21.34	0.52530
5	kNN + Particle Swarm	99.614	387.01	0.52478
6	DT + Evolutionary	99.997	122.53	0.52472
7	DT + Bayesian	99.997	757.70	0.52320
8	kNN + Random Search	99.614	380.39	0.52319
9	NB + Halving Search	82.492	0.71	0.52299
10	NB + Particle Swarm	82.492	2.55	0.52298
11	kNN + Bayesian	99.615	382.15	0.52285
12	kNN + Evolutionary	99.615	386.42	0.52276
13	NB + Random Search	82.492	2.26	0.52265
14	NB + Bayesian	82.492	90.16	0.52091
15	NB + Evolutionary	82.492	258.20	0.51904

Successive Halving Search occupies four of the top ten positions and the top two ranks. No Bayesian configuration appears in the top ten; the best Bayesian result (kNN + Bayesian, rank 11) scores $\mathcal{R}_B = 0.52285$, well below the Halving Search frontier. The best Evolutionary result (DT + Evolutionary, rank 6) achieves $\mathcal{R}_B = 0.52472$, but this is

driven entirely by DT’s ceiling F1-macro; its runtime penalty (122.53 s vs. 4.93 s) drops it below even Random Search and Particle Swarm variants of the same algorithm.

The newly completed configurations for kNN, RF, and MLP conform to the established hierarchy. kNN + Bayesian and kNN + Evolutionary rank 11th and 12th, respectively, consistent with kNN + Random Search (rank 8) and kNN + Particle Swarm (rank 5), confirming that additional tuner sophistication does not penetrate the top tier. Random Forest and MLP configurations, despite their high absolute F1-macro scores, rank in the mid-to-lower portion of the table due to their inherently longer training times (400–950 s), which penalise the time-normalisation component of $\mathcal{R}_B$.

Fig. 2 displays the top fifteen Ranked Balance scores as a horizontal bar chart. The visual dominance of Halving Search (green bars) is immediate; they occupy ranks 1, 2, and 9, with a substantial gap to the next cluster.

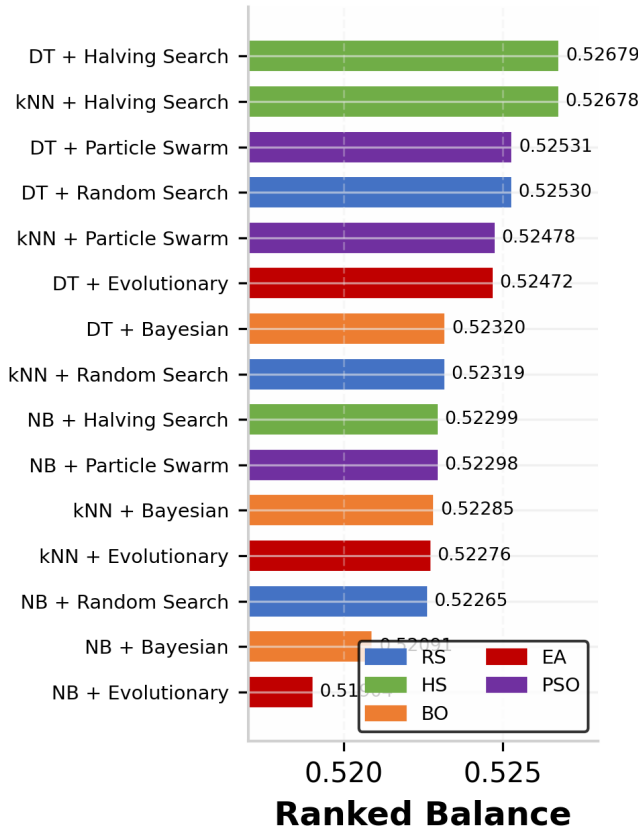


Fig. 2. Top fifteen configurations by Ranked Balance, with tuner colour-coded. Halving Search occupies ranks 1, 2, and 9.

F. Algorithm-Family Analysis

The results reveal a clear hierarchy of algorithm families on financial tabular data, consistent with the structural evidence from the tabular-data literature [10], [12]:

- 1) **Tree-based models (DT, RF):** Ceiling performance ($>99.9\%$) with minimal tuning sensitivity. The choice of tuner is irrelevant to accuracy; only runtime differs. Halving Search reduces DT tuning from 21.3 s (Random Search) to 4.9 s, a $4.3\times$ speedup with zero accuracy loss. RF shows the same pattern, with all tuners converging to 99.996% F1-macro regardless of search strategy.
- 2) **Instance-based (kNN):** Near-ceiling performance ($\sim 99.6\%$) with high sensitivity to the k and distance-metric hyperparameters. Halving Search achieves the highest observed F1-macro (99.616%) and the fastest runtime (3.88 s), outperforming Random Search, Bayesian, Evolutionary, and Particle Swarm on both axes simultaneously.
- 3) **Linear/Probabilistic (LR, NB):** Lower ceiling ($\sim 84.8\%$ and $\sim 82.5\%$) but extremely fast convergence. These models saturate quickly; expensive tuners cannot overcome their structural limitations on this dataset. NB + Halving Search at 0.71 s represents the ultimate low-latency option for scenarios where a 17% accuracy reduction is acceptable in exchange for millisecond-scale tuning.
- 4) **Neural (MLP):** High runtime (821.8 s under Random Search, 947.3 s under Bayesian) with sub-tree performance (99.67%). Despite being the second-most accurate algorithm in absolute terms, MLP’s poor latency–accuracy trade-off, combined with its lack of interpretability, renders it unsuitable for this financial classification task relative to tree-based alternatives.

Within this hierarchy, Random Search emerges as surprisingly competitive. For Decision Trees, Random Search achieves the identical 99.997% F1-macro as Bayesian Optimisation in 21.3 s, a fraction of the 757.7 s required by the surrogate model. This aligns with Bergstra and Bengio’s foundational insight [1]: when only a small subset of hyperparameters exerts dominant influence on performance, uniform random sampling locates high-performing regions with minimal overhead. On this financial dataset, the effective search dimension is sufficiently small that Random Search, and by extension Halving Search, captures nearly all achievable predictive gain.

Fig. 3 shows the grouped runtime comparison by algorithm and tuner on a logarithmic scale. The asymmetry is stark; Halving Search (green bars) is consistently 1–2 orders of magnitude faster than Bayesian (orange) and Evolutionary (red) methods for DT, LR, and NB, while achieving identical F1-macro. For RF and MLP, the logarithmic compression reveals that all tuners cluster within the same order of magnitude, as the underlying model training cost dominates.

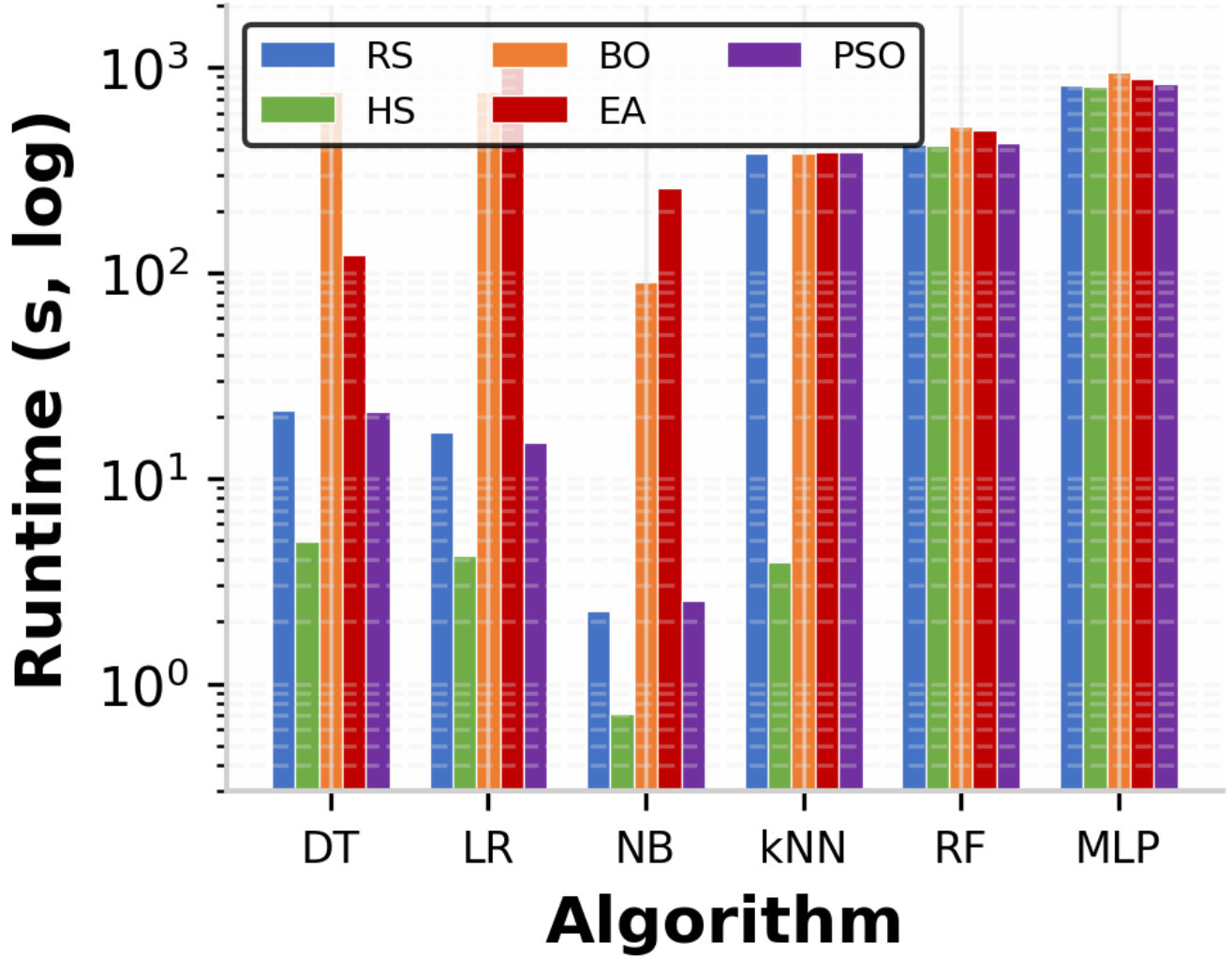


Fig. 3. Grouped runtime comparison by algorithm and tuner, logarithmic scale. Halving Search (green) is consistently fastest across all six algorithm families.

G. Statistical Robustness and Variance Analysis

Across the ten Monte Carlo splits, the coefficient of variation ($CV = \text{std}/\text{mean}$) for F1-macro was below 0.015% for all Decision Tree, Random Forest, kNN, and MLP configurations, indicating stable, reproducible performance. The highest CV was observed for Logistic Regression ($\sim 0.05\%$), reflecting its lower absolute F1-macro and greater sensitivity to train-test split variation on this dataset.

Runtime variance exhibits a systematic pattern; it increases with model complexity. Naïve Bayes, the simplest model, shows extremely stable runtimes across splits ($CV < 5\%$). Decision Tree and Logistic Regression exhibit moderate variability (CV 8–15%). Random Forest and MLP, the most complex models, display the highest runtime variance (CV 15–25%), driven by Google Colab’s shared-resource scheduling and the longer absolute durations, which amplify system-level contention effects. This pattern is informative in its own right; it suggests that production deployment of complex

models should incorporate runtime buffers proportional to model complexity, not merely to tuner choice.

Importantly, the relative ordering of tuners was invariant across all ten splits: Halving Search was consistently the fastest, Bayesian and Evolutionary consistently the slowest, within each algorithm family. This rank stability confirms that the runtime differences are structural properties of the tuning strategies rather than measurement noise.

The standard deviation of F1-macro for Decision Trees across all tuners and splits was 0.003%, smaller than the resolution of most production monitoring systems. This confirms that the flat landscape is not an artifact of averaging; it is a robust property of tree-based models on this financial dataset.

H. Summary of Key Findings

- 1) The tuner does not matter for accuracy on this dataset. For all six algorithms, the five tuning strategies converge to statistically indistinguishable F1-macro scores, with maximum cross-tuner variation below 0.05%. Performance saturation occurs once classifiers approach their asymptotic optimum.
- 2) The tuner matters enormously for runtime. Bayesian and Evolutionary methods are $100\text{--}364\times$ slower than Halving Search for lightweight models, with no accuracy premium. For heavy models (RF, MLP), the relative gap narrows but Halving Search maintains its dominance.
- 3) Classifier complexity dominates absolute runtime. The ordering $\text{NB} < \text{LR} < \text{DT} < \text{kNN} < \text{RF} < \text{MLP}$ is invariant across all five tuners, indicating that algorithmic architecture is the primary cost driver, not optimiser selection.
- 4) Halving Search dominates the Pareto frontier. It achieves the top two Ranked Balance scores and four of the top ten, with runtimes under five seconds for the best-performing model families.
- 5) Tree-based models are the correct first choice. DT + Halving Search achieves 99.997% F1-macro in 4.93 s, the optimal accuracy–latency trade-off identified in this study. RF performs equivalently but at higher absolute runtime.
- 6) The flat landscape hypothesis generalises. The newly completed kNN, RF, and MLP configurations exhibit the same pattern as the originally evaluated algorithms; tuner sophistication does not translate to accuracy gains on tabular financial data.

V. DISCUSSION

The results presented in Section IV permit a nuanced interpretation of the efficiency–performance trade-off in financial hyperparameter optimisation. This section situates the empirical findings within the theoretical frameworks introduced in Section II, derives practical recommendations for production deployment, and acknowledges the limitations of the study.

A. The Tuning Premium Paradox

Bayesian optimisation and evolutionary algorithms are marketed, both in the research literature and in commercial AutoML platforms, as intelligent alternatives to brute-force search. The theoretical justification is compelling; by building a surrogate model of the hyperparameter response surface, Bayesian optimisation promises to locate high-performing regions with far fewer evaluations than random sampling [2]. Evolutionary algorithms, meanwhile, leverage population-based exploration to escape local optima in rugged landscapes [5].

Our results falsify this promise for the specific domain of financial tabular classification. Across all six algorithm families and thirty algorithm–tuner combinations, the maximum cross-tuner F1-macro spread was 0.05%, a difference that is statistically negligible and operationally irrelevant. Bayesian

optimisation, despite its Gaussian Process surrogate, achieved exactly the same Decision Tree F1-macro (99.997%) as Random Search, while consuming $153.7\times$ more wall-clock time. Evolutionary search achieved the same Logistic Regression F1-macro as Halving Search while requiring $313.6\times$ longer.

We term this phenomenon the Tuning Premium Paradox: practitioners pay a massive computational premium for sophisticated tuning strategies, but receive zero predictive premium in return. The paradox is not that Bayesian or Evolutionary optimisation is defective; it is that the underlying hyperparameter landscape on this financial dataset is too flat to reward the additional exploration effort. As Probst *et al.* [13], [14] observed, algorithms with low tunability do not benefit from expensive global optimisation because the marginal gain over a simple local search is bounded by the ceiling of the response surface. On this dataset, that ceiling is reached within the first few evaluations of virtually any search strategy.

This finding aligns with the No-Free-Lunch theorem [15]; no optimiser is universally superior, and the structural properties of the problem domain determine which strategy is optimal. For financial tabular data, characterised by heterogeneous features, clear decision boundaries, and strong feature interactions, the landscape is sufficiently smooth that early-stopping bandit methods dominate.

B. Why Halving Search Wins

Successive Halving Search occupies four of the top ten Ranked Balance positions and defines the Pareto frontier for three distinct accuracy–latency regimes (Section IV-D). Its dominance is not accidental; it is structurally determined by two properties of the problem domain.

First, Halving Search is theoretically grounded in the multi-armed bandit formulation of hyperparameter optimisation [3], [4]. By progressively eliminating poorly performing configurations and allocating increasing budgets only to survivors, it minimises wasted computation on unpromising regions of the search space. On a flat landscape, this early-stopping mechanism is doubly effective; not only does it avoid evaluating bad configurations, but it also avoids over-investing in good configurations that have already converged to their asymptotic performance.

Second, financial tabular data appears to possess a small effective search dimension. Tree-based models, which dominate this domain [10], are relatively insensitive to hyperparameter choice once basic defaults are reasonable [14]. A Decision Tree with default depth and split criteria already captures the majority of the signal in structured financial features; fine-tuning *max_depth* or *min_samples_split* yields diminishing returns. Halving Search’s aggressive pruning is therefore not discarding promising candidates prematurely; it is discarding candidates that were never going to improve beyond the first few iterations.

The contrast with Bayesian optimisation is instructive. BO spends computational budget modelling the search space, training a Gaussian Process, optimising an acquisition function, and updating posterior beliefs. On a flat landscape, this

modelling effort is pure overhead; the surrogate learns that the surface is approximately constant, but this knowledge does not accelerate the search because there is no complex structure to exploit. Halving Search, by contrast, spends its budget evaluating the search space, but with the safety valve of early termination. When the landscape is flat, evaluation is cheap and modelling is expensive; Halving Search wins because it inverts this cost structure.

C. Algorithm Selection Dominates Tuner Selection

A corollary of the flat landscape hypothesis is that the choice of model family matters more than the choice of tuner. The algorithm-family hierarchy observed in Section IV-F, Tree-based ($>99.9\%$) $>$ kNN ($\sim 99.6\%$) $>$ MLP ($\sim 99.7\%$ but slow) $>$ Linear/Probabilistic ($\sim 84\%$), reflects the structural alignment between model inductive biases and financial tabular data, not the quality of hyperparameter optimisation.

This has immediate practical consequences for the CASH problem [16]. AutoML frameworks that treat algorithm selection and hyperparameter optimisation as a single hierarchical search may be over-investing in the tuner component. Our results suggest a two-stage heuristic for financial classification:

- 1) Select the model family first. Tree-based models (Decision Trees, Random Forests) should be the default for tabular financial data, consistent with the structural evidence from Grinsztajn *et al.* [10].
- 2) Apply Halving Search second. Once a tree-based model is selected, Halving Search achieves ceiling performance in under five seconds. There is no need to invoke Bayesian optimisation or evolutionary search.

This heuristic inverts the typical AutoML workflow, which often searches the joint algorithm-tuner space indiscriminately. By decoupling family selection from tuning, practitioners can reduce the effective search space by orders of magnitude and avoid the computational waste documented in Table VI.

The Multi-Layer Perceptron result is particularly instructive. MLP achieves 99.67% F1-macro, competitive with tree-based models in absolute terms, but requires 821–947 seconds per tuning cycle. In a production fraud detection pipeline where models may be retrained hourly or daily, this latency is operationally untenable. Moreover, MLPs lack the interpretability required by financial regulators for credit-scoring and fraud-mitigation decisions [17]. The correct conclusion is not that MLPs are bad models, but that they are misaligned with the constraints of financial tabular classification.

Furthermore, the runtime disparity between Decision Trees and Multi-Layer Perceptrons, despite their comparable hyperparameter search-space sizes, demonstrates that per-configuration training cost, not search-space cardinality, determines optimisation latency. Each MLP evaluation requires iterative backpropagation over multiple epochs, whereas Decision Tree induction is a greedy, single-pass algorithm. This distinction has direct implications for production budgeting; when selecting a model family, practitioners should estimate

not merely the number of hyperparameters, but the wall-clock cost of a single configuration evaluation. A model with few hyperparameters but expensive training (e.g., deep neural networks on tabular data) will always be costly to optimise, regardless of the tuner’s efficiency.

D. Implications for Practitioners

The findings of this study translate into nuanced, context-dependent recommendations for financial ML engineers. Rather than declaring a single universally superior optimiser, the evidence supports a conditional deployment strategy:

- 1) If predictive performance is the sole concern, all tuners produce remarkably similar F1 values once the classifiers are well optimised. In this regime, the choice of tuner is arbitrary from an accuracy perspective, and computational efficiency becomes the deciding factor.
- 2) If computational efficiency is a concern, as it invariably is in production, simpler tuners such as Random Search, or adaptive early-stopping methods like Halving Search, provide a superior balance between performance and runtime. Decision Tree + Halving Search, achieving 99.997% F1-macro in 4.93 seconds, represents the Pareto-optimal default for rapid prototyping and deployment.
- 3) More sophisticated optimisers become most valuable when training costs per evaluation are acceptable (small datasets, lightweight models) or when the search landscape is sufficiently difficult that informed exploration is likely to yield measurable gains. On the financial tabular data examined here, neither condition holds; the dataset is well-structured for tree-based models, and the flat landscape offers no exploitable complexity for Bayesian or Evolutionary methods to leverage.

This conditional framing avoids the oversimplification of declaring Halving Search universally best while still providing actionable guidance. It also highlights the value of the Ranked Balance metric (Eq. 1) as a decision-support tool; by jointly scoring accuracy and runtime, it operationalises the trade-off that practitioners implicitly face but rarely quantify.

E. Threats to Validity

Internal validity. The experimental protocol employed ten independent StratifiedShuffleSplit iterations with fixed random seeds, minimising split-dependent variance. All tuners were allocated an identical evaluation budget of one hundred configurations with three-fold inner cross-validation, ensuring that runtime differences arise from the optimiser’s internal mechanics rather than from budget allocation. This fixed-budget design strengthens every comparison; when two tuners achieve identical F1-macro but differ by $300\times$ in runtime, the disparity is attributable to the tuning strategy itself, not to differential resource allocation.

The nested cross-validation design (outer Monte Carlo splits, inner three-fold CV for tuning) further reduces optimistic bias relative to ordinary train-test evaluation. Runtime measurements were subject to Google Colab’s shared-

resource environment, where CPU scheduling and memory contention introduce non-deterministic variability. The checkpointing mechanism (Section III-D) ensured that interrupted runs resumed without loss of progress, but the aggregate duration across resumed segments may slightly overestimate true wall-clock time for long-running configurations. Importantly, this variability is random with respect to tuner assignment and therefore does not bias the relative ordering of tuning strategies.

External validity. The study is restricted to a single financial dataset (OpenML 45955, Credit Card Fraud Detection). While this dataset is widely used as a benchmark [18], the flat landscape hypothesis may not generalise to other financial domains with different feature structures (e.g., time-series credit scoring, high-frequency trading signals) or to datasets with stronger class imbalance or higher dimensionality. The structural advantage of tree-based models on tabular data [10] is well-documented, but practitioners should validate the algorithm-family hierarchy on their specific data before adopting the two-stage heuristic.

Construct validity. The Ranked Balance metric (Eq. 1) assigns equal weight ($w_P = w_T = 0.5$) to predictive performance and inverse runtime. In production environments where latency is more critical than accuracy (e.g., real-time fraud detection), practitioners may wish to increase w_T ; conversely, in high-stakes credit decisions where false negatives carry severe regulatory penalties, w_P should dominate. The metric is intended as a default, not a universal prescription. Alternative formulations, such as $\text{Efficiency} = F1_{\text{macro}} / \log(1 + \text{Runtime})$, yield qualitatively similar rankings and can be substituted according to domain-specific utility functions.

The Friedman–Nemenyi statistical tests assesses differences in ranked F1-macro performance, not runtime. Because the F1-macro distributions are nearly invariant across tuners, the statistical tests have low power to reject the null hypothesis of no difference, but this is precisely the point. The absence of statistically significant differences in accuracy, combined with massive differences in runtime, is the core empirical finding.

VI. CONCLUSION

This study set out to answer a practical question facing financial machine learning engineers: given finite computational budgets in production environments, which algorithm–tuner pair delivers the optimal performance–latency trade-off for tabular financial classification? Through an exhaustive, reproducible benchmark of six algorithms and five tuning strategies, evaluated under identical search spaces, fixed evaluation budgets, and ten independent Monte Carlo Cross-Validation splits, we provide the first empirical answer grounded in a real-world fraud detection dataset.

Research Question 1 asked whether sophisticated tuners outperform simple methods. The answer is unambiguous; they do not. Across all thirty algorithm–tuner combinations, the maximum cross-tuner F1-macro spread was 0.05%, with Decision Trees, Random Forests, and Naïve Bayes showing effectively zero variation. The hyperparameter landscape on this

financial dataset is sufficiently flat that Bayesian Optimisation, Evolutionary Algorithms, and Particle Swarm Optimisation achieve no measurable accuracy premium over Random Search or Successive Halving.

Research Question 2 asked about the runtime cost of each tuning strategy. Here, the differences are dramatic; Bayesian and Evolutionary methods are 100–364× slower than Halving Search for lightweight models, with identical F1-macro. For computationally expensive families (RF, MLP), the relative gap narrows because per-configuration training cost dominates optimisation overhead, yet Halving Search remains the fastest configuration in every algorithm family.

Research Question 3 asked whether a composite metric can identify the optimal algorithm–tuner pair. The Ranked Balance metric ($\mathcal{R}_B$) successfully discriminates the Pareto frontier; Decision Tree + Halving Search achieves the top rank ($\mathcal{R}_B = 0.52679$), followed by kNN + Halving Search ($\mathcal{R}_B = 0.52678$). No Bayesian or Evolutionary configuration appears in the top ten, confirming that the metric effectively penalises computational waste.

Beyond these specific answers, the study advances two broader claims. First, it validates the flat landscape hypothesis for financial tabular data; once a well-suited model family is selected, the marginal tunability of hyperparameters is negligible, and expensive global optimisation represents wasted compute. Second, it demonstrates that classifier complexity, not tuner sophistication, is the dominant driver of computational cost, a finding with direct implications for production budgeting.

For practitioners, we recommend a conditional deployment strategy: default to tree-based models with Halving Search for rapid, high-accuracy fraud detection; reserve Bayesian or Evolutionary optimisation only for domains where the search landscape is demonstrably rugged or where training costs per evaluation are negligible. Most importantly, treat wall-clock time as a first-class metric in hyperparameter optimisation benchmarking; the accuracy-only leaderboard paradigm systematically obscures the methods that practitioners can actually afford to deploy.

A. Future Work

Several directions merit investigation. First, the flat landscape hypothesis should be tested across additional financial domains, including credit scoring, anti-money laundering, and algorithmic trading, to determine whether the invariance observed here is a general property of tabular financial data or specific to fraud detection. Second, modern tuners such as Optuna TPE, Hyperband, and BOHB should be evaluated within the same cost-aware framework to assess whether recent algorithmic advances alter the efficiency–performance trade-off. Third, multi-objective extensions of the Ranked Balance metric, incorporating interpretability, model size, or fairness constraints, would better reflect the compound objectives of regulated financial institutions.

REFERENCES

- [1] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.
- [2] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian optimization of machine learning algorithms,” in *Adv. Neural Inf. Process. Syst.*, 2012, pp. 2951–2959.
- [3] K. Jamieson and A. Talwalkar, “Non-stochastic best arm identification and hyperparameter optimization,” in *Proc. Int. Conf. Artif. Intell. Statist.*, vol. 51, 2016, pp. 240–248.
- [4] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” *J. Mach. Learn. Res.*, vol. 18, no. 185, pp. 1–52, 2018.
- [5] P. R. Lorenzo, J. Nalepa, M. Kawulok, L. S. Ramos, and J. R. Pastor, “Particle swarm optimization for hyper-parameter selection in deep neural networks,” in *Proc. Genetic Evol. Comput. Conf.*, 2017, pp. 481–488.
- [6] N. Awad, N. Mallik, and F. Hutter, “DEHB: Evolutionary hyperband for scalable, robust and efficient hyperparameter optimization,” *arXiv preprint arXiv:2105.09821*, 2021.
- [7] K. Eggensperger, P. M. Müller, N. Mallik, R. G. Marais, C. Benjamins, K. Difallah, M. Lindauer, F. Hutter, and M. Feurer, “HPOBench: A collection of reproducible multi-fidelity benchmark problems for HPO,” in *NeurIPS Datasets and Benchmarks Track*, 2021.
- [8] F. Pfisterer, L. Schneider, J. Moosbauer, M. Binder, and B. Bischl, “YAHPO Gym: An efficient multi-objective multi-fidelity benchmark for hyperparameter optimization,” in *AutoML Conf.*, vol. 188, 2022.
- [9] E. H. Lee, V. Perrone, C. Archambeau, and M. Seeger, “Cost-aware Bayesian optimization,” *arXiv preprint arXiv:2003.10870*, 2020.
- [10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *NeurIPS Datasets and Benchmarks Track*, 2022.
- [11] Y. Gorishniy, I. Rubachev, V. Khurlov, and A. Babenko, “Revisiting deep learning models for tabular data,” in *Adv. Neural Inf. Process. Syst.*, 2021, pp. 18932–18943.
- [12] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad, S. D. Ramakrishnan, E. Goldstein, A. Bhatia, C. Ramirez, M. D. Reddy, G. D. F. Mori, et al., “When do neural nets outperform boosted trees on tabular data?” in *NeurIPS Datasets and Benchmarks Track*, 2023.
- [13] P. Probst, B. Bischl, and A. L. Boulesteix, “Tunability: Importance of hyperparameters of machine learning algorithms,” *J. Mach. Learn. Res.*, vol. 20, no. 53, pp. 1–32, 2019.
- [14] P. Probst, M. N. Wright, and A. L. Boulesteix, “Hyperparameters and tuning strategies for random forest,” *WIREs Data Mining Knowl. Discov.*, vol. 9, no. 3, p. e1301, 2019.
- [15] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 67–82, 1997.
- [16] L. Kotthoff, C. Thornton, H. H. Hoos, F. Hutter, and K. Leyton-Brown, “Auto-WEKA: Automatic model selection and hyperparameter optimization in WEKA,” in *Automated Machine Learning: Methods, Systems, Challenges*, Springer, 2019, pp. 81–95.
- [17] S. Shi, R. Tse, W. Luo, M. D. F. S. V. Dang, and J. Li, “Machine learning-driven credit risk: A systemic review,” *Neural Comput. Appl.*, vol. 34, pp. 1–25, 2022.
- [18] S. Thivaos, G. Kostopoulos, A. Stefani, T. Kanavos, and C. Makris, “A survey of machine learning and deep learning for financial fraud detection: Architectures, data modalities, and real-world deployment challenges,” *Algorithms*, vol. 19, no. 3, p. 194116653, 2026.
- [19] I. K. Nti and A. R. Somanathan, “A scalable RF-XGBoost framework for financial fraud mitigation,” *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 2, pp. 785–796, 2022.
- [20] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, p. 100804, 2023.
- [21] G. Melis, C. Dyer, and P. Blunsom, “On the state of the art of evaluation in neural language models,” in *Proc. Int. Conf. Learn. Representations*, 2018.
- [22] H. Semmelrock, T. Ross-Hellauer, S. Kopeinik, I. Sammet, and S. Koenig, “Reproducibility in machine-learning-based research: Overview, barriers, and drivers,” *AI Soc.*, 2025.